

LeetCode 2439: Minimize Maximum of Array

Authored by Arpit Raj, LNMIIT Jaipur | Date: July 14, 2026

Approach 1: Binary Search

Problem Observations

- Companies: PayTm, Oracle
- 0 index array comprising of n number
- In one operation we can pick any index (1 to n-1) and:
 - $nums[i] \rightarrow -1$
 - $nums[i-1] \rightarrow +1$
- O/P: minimum possible max integer
- So we can move elements to the left but never on the right
- If max 10 is possible then 11, 12, 13 are also possible.
- (min + monotonic) $\rightarrow$ Binary search

Binary search method is most understandable because it suspects an answer and whenever the suspicion is witness an answer greater than it, it tries to move excess to the left in hopes of reducing it.

Algorithm Logic

- $l = *min_element$ or 0 for simplicity
- $h = *max_element$
- We will process from right to left because we can move only to left.

Example: (3, 7, 1)

Let mid is 5

$1 \leq 5 \rightarrow$ do nothing

$7 > 5 \rightarrow$ excess = 2 so shift 2 pos

Resulting state: [5, 5, 1]

Time Complexity: $O(n \log(max_element))$

Implementation (Binary Search)

```
class Solution {
public:
    int n;
    bool check(vector<int>& nums,int mid){

        vector<long long> arr(nums.begin(),nums.end());
        for(int i=n-1;i>0;i--){
            if(arr[i]>mid){
                long long exc=arr[i]-mid;

                arr[i]-=exc; //aisa krne se arr[i] mid ke equal ho jaayega which is ok
                arr[i-1]+=exc;
            }
        }

        if(arr[0]<=mid){
            return true;
        }

        return false;
    }

    int minimizeArrayValue(vector<int>& nums) {
        n=nums.size();
        int l=*min_element(nums.begin(),nums.end());
        int h=*max_element(nums.begin(),nums.end());
        int ans=h;
        while(l<=h){

            int mid=l+(h-l)/2;

            if(check(nums,mid)){
                ans=mid;
                h=mid-1;
            }

            else{
                l=mid+1;
            }

        }

        return ans;
    }
};
```

Approach 2: Optimal $O(n)$

Concept (Prefix Sum)

Array: 3 7 1

Prefix Sum: 3 10 11

Then for every prefix, the minimum possible maximum must satisfy:

$$\text{maximum} \geq \text{ceil}(\text{prefix sum} / \text{prefix length})$$

Example Execution

- $ps = 10$
- $pl = 2$
- $10 / 2 = 5$

So one number must be ≥ 5 to be max.

i.e. one number must be greater or equal to mean to be classified as max.

Implementation (Optimal)

```
class Solution {
public:
    int minimizeArrayValue(vector<int>& nums) {
        int n=nums.size();
        vector<long long> pref(n+1,0);

        for(int i=1;i<=n;i++){
            pref[i]=pref[i-1]+nums[i-1];
        }
        int ans=INT_MIN;
        for(int i=1;i<=n;i++){
            int sus=(ceil)((double)pref[i]/i);
            ans=max(ans,sus);
        }

        return ans;
    }
};
```

